Data Storytelling & Interactive Dashboards

Suman 26 Feb, 2026 At 8:00 PM Pre-Reads Module-1 Recommended Resource

Associated Lectures (1)

Description Discussions

Session 5.2: Data Storytelling & Interactive Dashboards

Pre-read Material

Program: Vishlesan i-Hub IIT Patna x Masai School -- AIM (AI & Machine Learning) **Session ID:** 5.2 I **Week:** 5 **Prerequisites:** Session 5.1 (Matplotlib & Seaborn Fundamentals)

What You Will Learn Today

By the end of this session, you will be able to:

- **Explain the Context-to-Insight-to-Action framework** and distinguish between three levels of data communication: data dumps, chart collections, and data stories
- **Apply chart selection principles** to choose the right chart type for any analytical question, and identify common "chart crimes" (misleading axes, chart junk, wrong chart types)
- **Create interactive visualizations** using Plotly Express, including line charts, bar charts, scatter plots, and animated bubble charts with hover tooltips and zoom capabilities
- **Build your first web application** using Streamlit that displays data, metrics, and interactive charts entirely in Python (no HTML, CSS, or JavaScript required)
- **Add interactive filter widgets** (selectbox, multiselect, slider) to a Streamlit app and understand how Streamlit's reactive execution model works
- **Design professional dashboard layouts** using Streamlit's columns and sidebar, applying the narrative flow principle (Summary → Trend → Breakdown → Detail)
- **Assemble a complete E-commerce Interactive Dashboard** as a Streamlit application with sidebar filters, KPI metric cards, and multiple Plotly charts that update in real time

Business Story: The Presentation That Changed How the World

Sees Global Development

In February 2006, a Swedish physician named Hans Rosling walked onto the TED stage and gave a presentation that would be viewed over 14 million times, becoming one of the most influential data visualizations in history. The title: **"The Best Stats You've Ever Seen."**

Rosling's audience included United Nations diplomats, policy makers, philanthropists, and global health experts—people who made decisions affecting millions of lives. He started by polling them with simple questions: "Which country has higher child mortality—Sri Lanka or Turkey?" "What percentage of the world's population lives in low-income countries?"

The results were shocking. These educated, well-informed experts performed **worse than random chance**. A chimpanzee picking answers randomly would have scored higher. Their mental models of the world were based on data from the 1960s and 1970s—decades out of date.

Then came the reveal.

Rosling showed an **animated bubble chart**. Two hundred countries, each represented as a bubble. The x-axis showed income (GDP per capita). The y-axis showed life expectancy. The size of each bubble represented population. The color indicated continent. Then he pressed play—and the bubbles began moving through time, from 1800 to 2006.

For four minutes, the audience watched in stunned silence. Country after country moved from the "poor and sick" corner toward the "rich and healthy" zone. China's massive bubble surged forward. India followed. Bangladesh—a country the audience assumed was hopelessly behind—moved steadily upward and to the right. The gap between "developing" and "developed" countries that everyone believed was widening was actually **closing**.

The audience erupted in applause—**mid-presentation**. That almost never happens at TED.

The Devastating Insight

Here's the thing: **the data was not new.** The World Health Organization and World Bank had been publishing these statistics for years. Every person in that audience could have found these datasets online. But nobody had **looked** at them—because nobody had **presented** them in a way that was compelling enough to challenge deeply held assumptions.

Rosling did not discover new data. He told a **story** with existing data. And that story changed minds at the highest levels of global policy.

Google acquired Rosling's visualization tool (Gapminder Trendalyzer) in 2007 and integrated it into Google Public Data Explorer. Rosling's work proved three fundamental truths:

> **Data that is not presented compellingly might as well not exist**
>
> **Interactive, animated visualization can communicate complex multi-dimensional trends that static charts cannot**
>
> **The right visualization can change minds and drive policy at the highest levels**

The Lesson for You

In Session 5.1, you learned to create beautiful static visualizations with Matplotlib and Seaborn. You can make line charts, bar charts, scatter plots, histograms, heatmaps, and multi-panel figures exported at 300 DPI. That is real, portfolio-quality work.

But every chart you built is a **frozen image**. Once you export it as a PNG file, it's done. No interaction. No exploration. No filtering. It communicates ONE story from ONE perspective, and the viewer either accepts that perspective or moves on.

Today's session addresses that limitation head-on. You will learn:

- **The PRINCIPLES of data storytelling** — because even an interactive dashboard is useless if it doesn't tell a coherent story
- **The TOOLS for interactivity** — Plotly Express for interactive charts and Streamlit for building complete web-based dashboards entirely in Python

By the end of today, you will build a live, interactive dashboard that runs in a web browser, has filter controls in a sidebar, displays KPI metrics, and contains multiple charts that all update in real time when the user changes a filter. **And you will do it entirely in Python—no HTML, no CSS, no JavaScript.**

Setup Requirements

This session introduces two new libraries: **Plotly** (for interactive visualizations) and **Streamlit** (for building web applications in Python). Both must be installed before class.

Step 1: Install Required Libraries

Open your terminal (Command Prompt on Windows, Terminal on Mac/Linux) and run:

```
pip install plotly streamlit
```

Expected output: You should see messages indicating successful installation of both packages and their dependencies.

Step 2: Verify Plotly Installation

Open a Python interpreter or Jupyter notebook and run:

```
import plotly.express as px
print(f"Plotly version: {px.__version__}")

# Quick test: create a simple interactive scatter plot
import pandas as pd
df = pd.DataFrame({'x': [1, 2, 3, 4], 'y': [10, 20, 15, 25]})
fig = px.scatter(df, x='x', y='y', title='Plotly Test Chart')
fig.show()
```

Expected result: A new browser tab should open displaying an interactive scatter plot. You should be able to hover over points to see their coordinates, and use the toolbar to zoom and pan.

Step 3: Verify Streamlit Installation

IMPORTANT: Streamlit applications **cannot run inside Jupyter notebooks**. They must be saved as `.py` files and launched from the terminal.

Create a test file named `test_streamlit.py` with the following content:

```
import streamlit as st

st.title("Streamlit Test App")
st.write("If you can see this, Streamlit is working!")
st.metric("Test Metric", value=42)
```

Save the file, then run from the terminal:

```
streamlit run test_streamlit.py
```

Expected result: A new browser tab should open at `http://localhost:8501` showing your test app with a title, text, and a metric card displaying the number 42.

To stop the Streamlit app: Press `Ctrl+C` in the terminal.

Step 4: Review Session 5.1 Materials

Make sure you are comfortable with:

- Matplotlib's object-oriented interface (`fig, ax = plt.subplots()`)
- Creating line charts, bar charts, scatter plots, and histograms
- Customizing charts with titles, labels, legends, and color palettes
- Seaborn statistical plots (boxplot, heatmap, countplot, pairplot)

You will build on these skills today by adding interactivity and web-based presentation.

Concepts to Preview

1. Data Storytelling Principles

The Three Levels of Data Communication:

Most analysts operate at Level 1 or Level 2. The ones who get promoted reach Level 3.

- **Level 1: Data Dump** (lowest impact) Raw tables, spreadsheets, CSV exports sent via email. The reader must do ALL the interpretive work. Result: Nobody reads it. It sits in an inbox, unopened.
- **Level 2: Chart Collection** (moderate impact) Multiple visualizations arranged on a page or dashboard. Looks professional—charts are technically correct and well-formatted. BUT: no narrative thread connecting them, no interpretation, no recommendation. Result: The viewer thinks "interesting" and moves on without changing a decision.

- **Level 3: Data Story** (highest impact) Data + Narrative + Visuals, structured to guide the audience from context through insight to action. Every chart supports a specific argument. The headline states the insight, not just the chart title. Ends with a clear recommendation. Result: The audience understands, agrees, and acts.

The Context → Insight → Action Framework:

Every effective data presentation—whether a one-slide executive summary, a 20-page report, or an interactive dashboard—answers three questions:

CONTEXT: "Why are we looking at this?" The business problem, question, or decision that requires data. Sets the stage—tells the audience WHY they should care. *Example: "Our e-commerce return rate has increased from 8% to 12% over the past quarter, costing an additional INR 45 lakh per month in reverse logistics."*

INSIGHT: "What did the data reveal?" The specific finding, pattern, or anomaly discovered through analysis. Must be specific and quantified. *Example: "Analysis shows that 67% of returns come from three product categories where size guides were removed in the website redesign."*

ACTION: "What should we do about it?" The clear recommendation or decision the data supports. Must be actionable. *Example: "Reinstating size guides for these three categories is projected to reduce returns by 40%, saving approximately INR 2.3 crore annually."*

The Inverted Pyramid (Dashboard Narrative Flow):

Borrowed from journalism, this principle structures dashboards so that the most important information appears first:

SUMMARY (Top): KPI metric cards—the 3-5 most important numbers (total revenue, total orders, average rating). These tell the story in 5 seconds.

TREND (Second row): Line charts showing how the KPIs change over time. These answer "is it getting better or worse?"

BREAKDOWN (Third row): Bar charts or pie charts showing composition (revenue by category, orders by city). These answer "where specifically?"

DETAIL (Bottom): Filtered data tables, scatter plots, or drill-down views for analysts who need to dig deeper.

Design principle: A VP should get the answer from the top row. A manager should get the answer from the top two rows. An analyst should explore all four rows.

---

2. Chart Selection & Design

The Chart Selection Decision Tree:

Before you write any plotting code, ask yourself: **"What analytical question am I trying to answer?"** The answer determines the chart type.

Question	Chart Type
How does Y change over time?	**Line chart** (time on x-axis, measured value on y-axis)

Question	Chart Type
How do categories compare?	**Bar chart** (horizontal if many categories for readability)
Is there a relationship between X and Y?	**Scatter plot** (each point is one observation)
What is the distribution of X?	**Histogram** (for shape) or **Boxplot** (for median, quartiles, outliers)
What is the composition of the whole?	**Pie chart** ⚠️ (almost always better as a bar chart)
How do multiple variables correlate?	**Heatmap** or **Pairplot**

The Three Chart Crimes:

Crime #1: Misleading Axes

- Truncated y-axis starting at a non-zero value to exaggerate changes
- Reversed or reordered axes to fake trends (remember the Georgia COVID chart)
- Dual y-axes with different scales making unrelated trends appear correlated

Crime #2: Chart Junk (Edward Tufte's term)

- 3D effects on 2D data that distort area perception
- Excessive gridlines that compete with data for attention
- Unnecessary decorations: gradient backgrounds, clip art, logos overlapping data
- **The rule:** If a visual element doesn't encode DATA, it should probably be removed

Crime #3: Wrong Chart Type

- Pie chart with 15 categories (impossible to compare slices visually)
- Line chart for categorical data (lines imply continuity that doesn't exist)
- Bar chart for distribution (bars suggest discrete counts, obscuring the continuous shape)

The Data-Ink Ratio (Edward Tufte):

Maximize the proportion of ink (or pixels) devoted to displaying actual data. Minimize everything else.

What to REMOVE:

- Top and right borders (spines) — they frame the plot but add no information
- Unnecessary gridlines
- Background colors and gradients
- 3D effects
- Legends when direct labels suffice

What to ADD:

- Value labels directly on bars or points
- Annotations on key data points

- Strategic color accents (gray for baseline, RED for the insight point)

Color Theory Essentials:

- **Sequential palette:** Light → dark for continuous values (temperature, revenue). Use when values have a natural order from low to high.
- **Diverging palette:** Two colors radiating from a neutral center (blue ← white → red) for profit/loss, above/below average. Use when there's a meaningful midpoint.
- **Categorical palette:** Distinct colors for discrete groups (blue, orange, green). Use when colors represent different GROUPS, not different MAGNITUDES.

CRITICAL: Colorblindness Accessibility

- ~8% of men and ~0.5% of women have color vision deficiency (most commonly red-green colorblindness)
- In a meeting of 100 people, statistically 4-8 cannot distinguish red from green
- **Rules:**
  - NEVER rely on color alone to convey information—add labels or patterns
  - AVOID red-green combinations as primary contrast
  - Use colorblind-friendly palettes (Seaborn's `"colorblind"` palette, Plotly's built-in accessible scales)
  - Test your charts by converting to grayscale—if you lose information, your color scheme is fragile

---

3. Interactive Visualization with Plotly Express

What is Plotly Express?

Plotly Express (`px`) is a high-level Python library for creating interactive visualizations. Unlike Matplotlib (which produces static images), Plotly charts:

- Display **hover tooltips** showing exact data values when you mouse over points
- Support **zoom and pan** to explore specific regions of the chart
- Include a **toolbar** for downloading, selecting, and customizing the view
- Can be **exported as HTML files** that work in any web browser (no Python required)
- Support **animations** (like Rosling's famous animated bubble chart)

Basic Plotly Express syntax:

```
import plotly.express as px

# Line chart
fig = px.line(df, x='date', y='revenue', title='Revenue Over Time')
fig.show()

# Bar chart
fig = px.bar(df, x='category', y='revenue', title='Revenue by Category')
fig.show()

# Scatter plot
fig = px.scatter(df, x='price', y='rating', color='category',
```

```
                  size='quantity', title='Price vs Rating')
fig.show()

# Animated bubble chart (Rosling-style)
fig = px.scatter(df, x='gdp_per_capita', y='life_expectancy',
                  size='population', color='continent',
                  animation_frame='year', animation_group='country',
                  title='Global Development 2000-2020')
fig.show()
```

Key differences from Matplotlib:

- Plotly Express uses **DataFrame column names** directly (no `df['column']` extraction needed)
- The chart appears in a **new browser tab** instead of inline in Jupyter
- Charts are **interactive by default** (no additional configuration required)
- Export with `fig.write_html('chart.html')` for standalone HTML files

---

4. Building Web Applications with Streamlit

What is Streamlit?

Streamlit is a Python library that turns Python scripts into **interactive web applications**. You write a `.py` file, run it from the terminal with `streamlit run filename.py`, and Streamlit launches a live web server displaying your app in a browser.

No HTML. No CSS. No JavaScript. Pure Python.

Basic Streamlit components:

```
import streamlit as st
import pandas as pd

# Display text
st.title("My Dashboard")
st.header("Section Header")
st.write("This is regular text")

# Display data
df = pd.DataFrame({'A': [1, 2, 3], 'B': [10, 20, 30]})
st.dataframe(df)  # Interactive table with sorting

# Display metrics (KPI cards)
st.metric(label="Total Revenue", value="INR 7.3L", delta="+12%")

# Display Plotly charts
import plotly.express as px
fig = px.line(df, x='A', y='B')
st.plotly_chart(fig)
```

Streamlit's Reactive Execution Model:

This is the most important concept to understand:

- **The entire script re-runs from top to bottom every time any widget value changes**
- When a user selects a different value from a dropdown, the script runs again with the new value
- All variables are recalculated, all charts are regenerated
- This "reactive" model means you don't need to write callbacks or event handlers—Streamlit handles it automatically

Interactive widgets:

```
# Dropdown (single selection)
category = st.selectbox("Choose category", ["Electronics", "Clothing", "Books"])

# Multiselect (multiple selections)
cities = st.multiselect("Choose cities", ["Mumbai", "Delhi", "Bangalore"])

# Slider
price_range = st.slider("Price range", min_value=0, max_value=10000, value=(1000, 5000))

# Date input
start_date = st.date_input("Start date")

# The selected values are stored in the variables (category, cities, price_range, start_da
# Use them to filter your DataFrame and generate charts
```

Layout with sidebar and columns:

```
# Sidebar (for filters and controls)
with st.sidebar:
    st.header("Filters")
    category = st.selectbox("Category", ["All", "Electronics", "Clothing"])

# Main area with columns
col1, col2, col3 = st.columns(3)
with col1:
    st.metric("Revenue", "INR 7.3L")
with col2:
    st.metric("Orders", "40")
with col3:
    st.metric("Avg Rating", "3.8")
```

The workflow:

Write your Streamlit app in a `.py` file (e.g., `dashboard.py`)

Save the file

Open terminal and run: `streamlit run dashboard.py`

Browser opens at `http://localhost:8501` with your live app

Edit the `.py` file and save—Streamlit auto-reloads the app

Press `Ctrl+C` in terminal to stop the server

---

5. Dashboard Design Patterns

The Mini-Project Preview:

Today you will build a complete **E-commerce Interactive Dashboard** with:

- **Sidebar filters:** Product category (multiselect), city (multiselect), date range
- **KPI metrics row:** Total revenue, total orders, average order value, average rating
- **Plotly line chart:** Monthly revenue trend
- **Plotly bar chart:** Revenue by category (horizontal, sorted)
- **Plotly scatter plot:** Price vs rating, colored by category

When the user changes any filter in the sidebar, **all metrics and charts update instantly**.

Dashboard design principles you'll learn:

- Apply the inverted pyramid structure (summary first, detail last)
- Use color strategically (gray for context, red accent for insights)
- Design for two audiences: executives (who read the top row only) and analysts (who explore everything)
- Write insight headlines, not just chart titles ("Electronics Leads at INR 2.45L" not "Revenue by Category")

Key Vocabulary

Term	Definition
Data Storytelling	Combining data, narrative, and visuals to guide an audience from context through insight to action
Context-Insight-Action Framework	A three-step structure for data presentations: (1) Why does this matter? (2) What did we find? (3) What should we do?
Data Dump	Level 1 communication: raw tables with no interpretation (lowest impact)
Chart Collection	Level 2 communication: multiple charts without a narrative thread (moderate impact)
Data Story	Level 3 communication: charts structured around a coherent argument with recommendations (highest impact)
Inverted Pyramid	A narrative structure that presents the most important information first (borrowed from journalism)
Chart Junk	Edward Tufte's term for visual elements that do not encode data (3D effects, gradients, excessive gridlines)
Data-Ink Ratio	The proportion of ink (or pixels) devoted to displaying actual data versus decorative elements (goal: maximize this ratio)
Truncated Y-Axis	A misleading chart crime where the y-axis starts at a non-zero value to exaggerate changes

Term	Definition
Colorblind-Friendly Palette	A color scheme that remains distinguishable for people with color vision deficiency (avoids red-green combinations)
Sequential Palette	Colors progressing from light to dark, used for continuous data with a natural order (low to high)
Diverging Palette	Two colors radiating from a neutral center, used for data with a meaningful midpoint (negative ← 0 → positive)
Categorical Palette	Distinct colors representing different groups with no inherent order
Interactive Visualization	A chart that responds to user input (hover, zoom, pan, filter) rather than displaying a static image
Hover Tooltip	A small popup that appears when you mouse over a data point, showing its exact values
Animated Chart	A visualization that changes over time (e.g., Rosling's bubble chart moving through years)
Plotly Express	A high-level Python library for creating interactive visualizations with minimal code
Streamlit	A Python framework for building interactive web applications entirely in Python (no HTML/CSS/JavaScript)
Reactive Execution Model	Streamlit's behavior of re-running the entire script from top to bottom whenever any widget value changes
Widget	An interactive UI element (dropdown, slider, checkbox) that captures user input
Sidebar	A Streamlit layout component that displays filters and controls on the left side of the screen
KPI (Key Performance Indicator)	A critical metric that summarizes business performance (e.g., total revenue, average order value)
Metric Card	A visual component displaying a single KPI value with optional delta (change) indicator
Dashboard Narrative Flow	The sequence in which information is presented: Summary → Trend → Breakdown → Detail

Things to Ponder

Before class, think deeply about these questions:

> **On data influence:** Think of a time when a chart or data visualization changed your mind about something—maybe a news infographic, a COVID dashboard, or a social media data post. What made it compelling? Was it the data itself, or how it was presented?

On misleading charts: Why do you think chart crimes (truncated axes, misleading scales, wrong chart types) are so common? Are they always intentional deception, or do they sometimes happen due to ignorance or carelessness? How can you ensure your charts are honest?

On static vs interactive: When should you use a static PNG chart (like those from Matplotlib), and when should you build an interactive dashboard? What are the trade-offs? Hint: Think about audience, distribution method, and level of exploration needed.

On the "so what?" problem: Why do most corporate dashboards fail to drive action? You've probably seen dashboards—in school, at work, online—that display lots of charts but don't lead to any decision. What separates a dashboard people glance at from a dashboard people act on?

On the Rosling effect: The WHO and World Bank published global development data for decades, but nobody paid attention until Rosling presented it compellingly. What does this say about the role of presentation in data science? Is being "right" enough, or do you also need to be "persuasive"?

On accessibility: If 8% of men are colorblind, why do so many dashboards still use red-green color schemes? What responsibilities do data professionals have to ensure their visualizations are accessible to everyone?

On dashboard audience: Should a dashboard be designed for executives (who want the answer in 5 seconds) or analysts (who want to explore deeply)? Or can one dashboard serve both? How would you achieve that?

Optional Deep Dive

If you want to explore these topics further before class, here are some curated resources:

Watch

- **Hans Rosling's 2006 TED Talk: "The Best Stats You've Ever Seen"** https://www.ted.com/talks/hans_rosling_the_best_stats_you_ve_ever_seen (20 minutes — HIGHLY RECOMMENDED. This is the visualization that started a revolution.)
- **Hans Rosling's follow-up: "200 Countries, 200 Years, 4 Minutes"** https://www.youtube.com/watch?v=jbkSRLYSojo (4 minutes — The condensed, pure visualization version)

Read

- **Edward Tufte: "The Visual Display of Quantitative Information"** (Book) The definitive text on data visualization principles. Focus on the chapter about chart junk and the data-ink ratio.
- **Storytelling with Data by Cole Nussbaumer Knaflic** (Book) Excellent practical guide to choosing charts, decluttering, and crafting narratives. Highly accessible for beginners.
- **The Functional Art by Alberto Cairo** (Book) Bridges the gap between data journalism and data science. Excellent chapter on misleading visualizations.

Explore

- **Gapminder Foundation Website** https://www.gapminder.org/ Explore the actual Gapminder tools and data. Play with the bubble chart yourself.
- **Streamlit Gallery** https://streamlit.io/gallery Browse hundreds of real Streamlit dashboards built by the community. Get inspiration for your own projects.
- **Plotly Express Documentation** https://plotly.com/python/plotly-express/ Official examples of every chart type available in Plotly Express.
- **Color Brewer (Colorblind-Safe Palettes)** https://colorbrewer2.org/ An interactive tool for selecting colorblind-friendly palettes for data visualization.
- **How to Lie with Statistics by Darrell Huff** (Book) A classic 1954 book on statistical manipulation and misleading charts. Still relevant today.

Practice (If You Have Time)

- Create a simple Streamlit app that displays your name, a metric, and a Plotly line chart. Get comfortable with the workflow: write `.py`, run `streamlit run`, see results in browser.
- Find a chart online (news article, social media, corporate report) and critique it: What question does it answer? What chart type is used? Is it honest or misleading? How would you redesign it?

Pre-Class Checklist

Use this checklist to ensure you're ready for the session:

- [ ] **Installed Plotly** — Run `pip install plotly` and verify with `import plotly.express as px`
- [ ] **Installed Streamlit** — Run `pip install streamlit` and verify with `streamlit run test_streamlit.py`
- [ ] **Tested Plotly interactivity** — Created a simple `px.scatter()` chart and confirmed it opens in browser with hover tooltips
- [ ] **Tested Streamlit workflow** — Created a `.py` file, ran `streamlit run filename.py`, saw app in browser, stopped server with `Ctrl+C`
- [ ] **Reviewed Session 5.1 materials** — Comfortable with Matplotlib's `fig, ax = plt.subplots()` pattern and Seaborn statistical plots
- [ ] **Watched Hans Rosling's TED Talk** (recommended but optional) — Experienced the "Rosling moment" firsthand
- [ ] **Thought about misleading charts** — Identified at least one example of a chart crime from your own experience (news, social media, work)
- [ ] **Identified a personal data story** — Recalled a time when a visualization changed your mind or influenced your decision
- [ ] **Set up workspace** — Have both a Jupyter environment (for Plotly testing) AND a text editor or VS Code (for writing Streamlit `.py` files)
- [ ] **Cleared mental space** — Ready to shift from static visualization (Session 5.1) to interactive storytelling (Session 5.2)

Final Thought

In Session 5.1, you learned to create beautiful static charts. Today, you level up.

You will learn to tell **stories** with data—to structure your analysis so that it changes minds and drives action. You will learn to build **interactive experiences** that let your audience explore data themselves. And you will learn to deploy **web applications** that turn your Python code into tools that anyone can use in a browser.

By the end of today, when someone asks you "Can you build a dashboard?", your answer will be: **"Yes. And I can deploy it as a web app."**

That is not a trivial skill. That is career-changing capability.

See you in class.

Vishlesan i-Hub IIT Patna × Masai School